Design and Deployment of a Customer Data Processing System on AWS

Maab Osman

14.05.2026

Table of Contents

Introduction

This project focuses on the design and deployment of a lightweight customer data processing system on AWS. The goal of the project is to create a small but realistic cloud architecture that supports storing, processing, and managing customer-related data. In Phase 1, the focus is on building the core infrastructure by using EC2, RDS, and S3.

The chosen topic supports my interest in cloud infrastructure and backend-oriented systems. Instead of building only a simple website, I wanted to create a more business-relevant architecture that reflects how real organizations process and store customer data. The project is designed in a way that allows Phase 2 to extend the solution with cloud management and governance features such as monitoring, alerting, logging, and cost optimization.

The main objective of Phase 1 is to deploy the infrastructure, validate connectivity between the services, and document the implementation so that it can be recreated in the AWS Learner Lab environment.

Project Plan

Phase 1 Design

The Phase 1 architecture is designed as a multi-tier cloud environment for customer data processing. The system is deployed inside a Virtual Private Cloud (VPC) with clearly separated public and private subnets.

The EC2 instance is placed in a public subnet and acts as the application and processing layer of the system. It is responsible for interacting with other AWS services and performing validation tasks. The RDS database is deployed in private subnets across multiple Availability

Zones using a DB subnet group. This ensures that the database is not publicly accessible and follows secure design principles.

S3 is used as an object storage service for storing raw customer data files and potential exports. The EC2 instance communicates with S3 to upload and retrieve files.

Security groups are configured to restrict access between components. The RDS instance only allows inbound traffic on port 3306 from the EC2 security group, ensuring that only the application layer can access the database. Internet access to the EC2 instance is enabled through an Internet Gateway and a public route table.

The architecture of the system is illustrated in Figure 1 below.

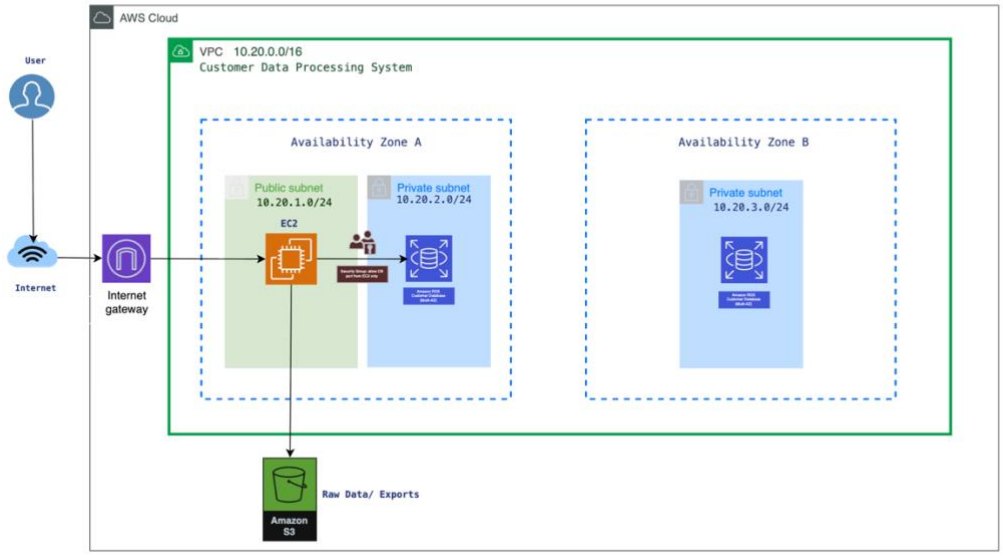


Figure 1: Architecture of the Customer Data Processing System

This architecture was chosen because it is small enough to implement within the limits of the AWS Academy Learner Lab environment, but still realistic enough to reflect a common business use case. It also provides a strong foundation for Phase 2, where governance

features such as monitoring, logging, and alerting can be added on top of the deployed environment.

Phase 2 Design

In Phase 2, cloud governance and management features will be added to the existing architecture to improve observability, security, and cost efficiency.

Monitoring will be implemented using Amazon CloudWatch to track EC2 and RDS performance metrics such as CPU utilization and database connections. Alerts will be configured using Amazon SNS to notify when predefined thresholds are exceeded.

Logging will be enabled using AWS CloudTrail to track API activity and system changes. Additionally, application-level logging can be integrated with CloudWatch Logs.

Security improvements will include the use of IAM roles for EC2 instances to securely access AWS services such as S3 without relying on preconfigured credentials.

Cost optimization strategies will be considered by analyzing resource utilization and ensuring that only necessary services are running.

Implementation

Phase 1 Implementation and Validation

The goal of Phase 1 was to deploy the planned customer data processing architecture in AWS and verify that all components function correctly and can communicate with each other.

Resources Created

The following AWS resources were created:

- Virtual Private Cloud
- One public subnet
- Two private subnets in different Availability Zones
- Internet Gateway
- Public route table
- Amazon EC2 instance
- Amazon RDS instance
- DB subnet group
- Amazon S3 bucket
- Security groups for EC2 and RDS

Networking Setup

The networking components were created, including the VPC, subnets, Internet Gateway, and route table. The public subnet was configured with internet access.

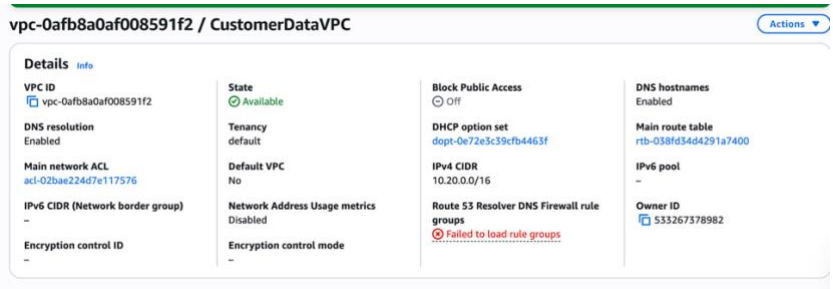


Figure 2: VPC Configuration

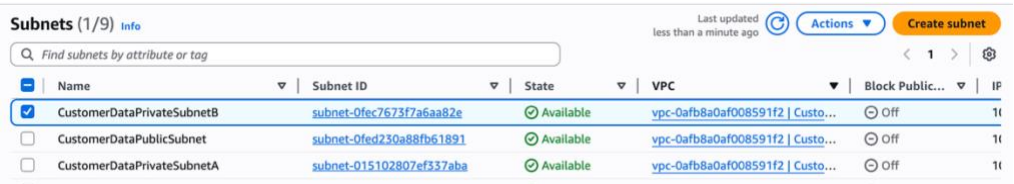


Figure 3: Subnets Configuration

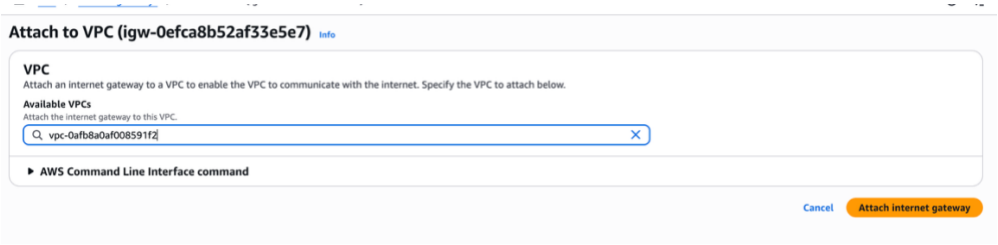


Figure 4: Internet Gateway attachment

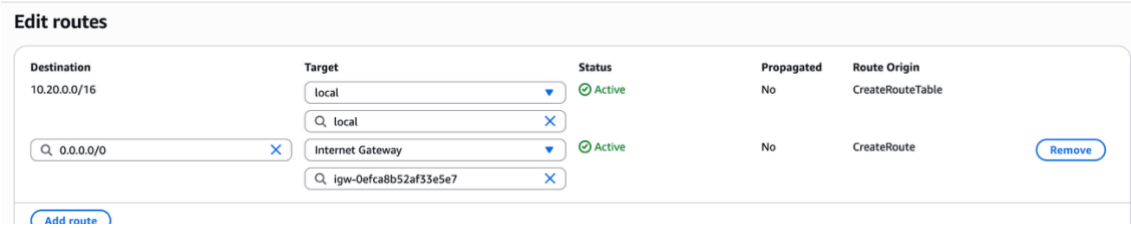


Figure 5: Route table configuration

Storage Configuration

An S3 bucket was created to store customer data files. The functionality was validated by uploading and listing a sample CSV file.

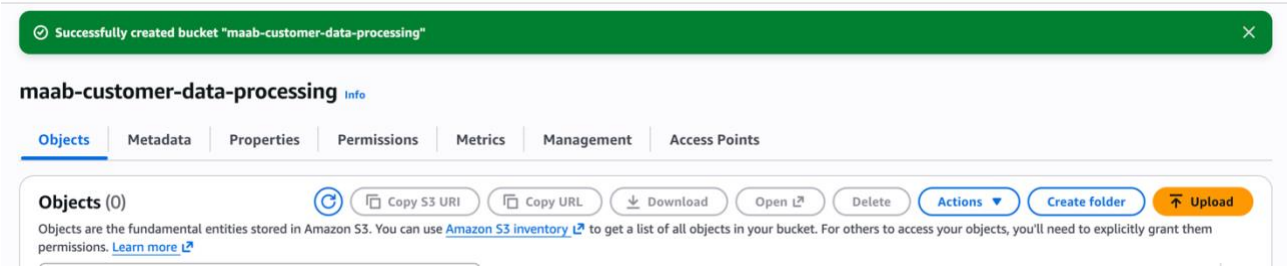


Figure 6: S3 Bucket

```
eee_W_6011404@runweb219702:~$ cat > customers.csv << 'EOF'
> id,name,email
> 1,Ali Khan,ali@example.com
> 2,Sara Ahmed,sara@example.com
> 3,Omar Hassan,omar@example.com
> EOF
eee_W_6011404@runweb219702:~$ aws s3 cp customers.csv s3://maab-customer-data-processing/
upload: ./customers.csv to s3://maab-customer-data-processing/customers.csv
eee_W_6011404@runweb219702:~$ aws s3 ls s3://maab-customer-data-processing/
2026-03-31 08:55:28        102 customers.csv
eee_W_6011404@runweb219702:~$
```

Figure 7: S3 file upload validation

Security Configuration

Security groups were configured to control access between system components. The RDS instance allows database access only from the EC2 security group.

Additionally, the principle of least privilege should be applied when configuring IAM roles and security groups to minimize unnecessary access.

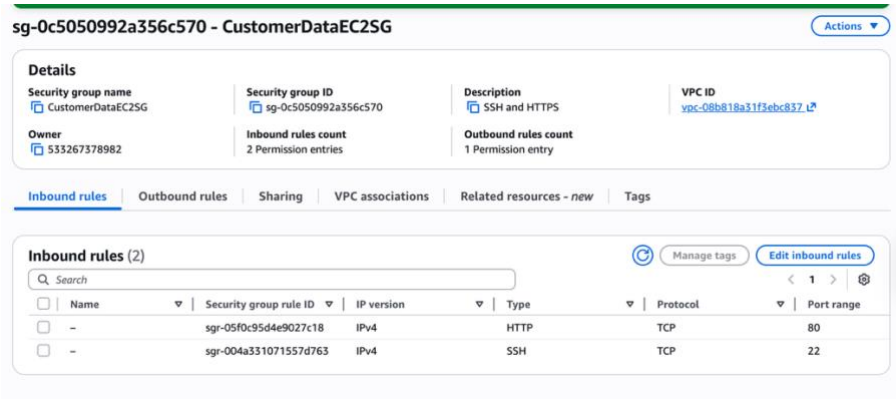


Figure 8: EC2 security group

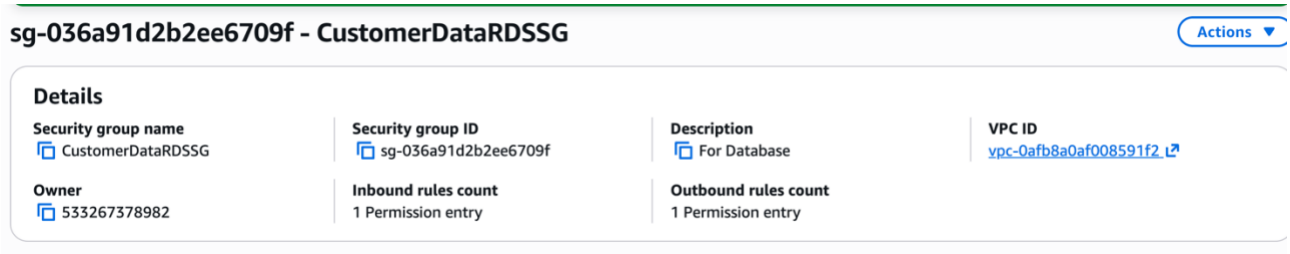


Figure 9: RDS Security Group

Compute and Database Deployment

An EC2 instance was deployed in the public subnet to act as the processing layer. The RDS database was deployed in private subnets using a DB subnet group.

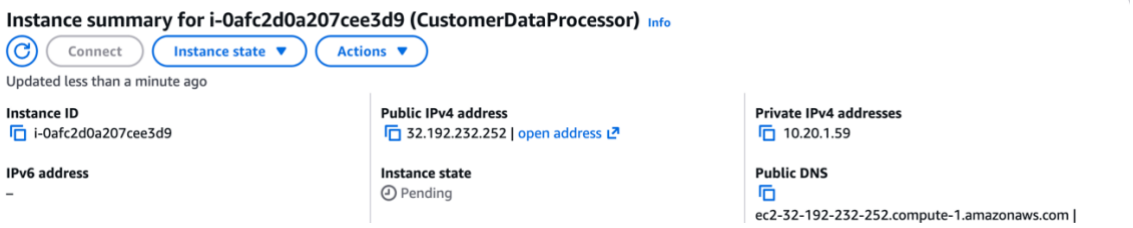


Figure 10: EC2 instance

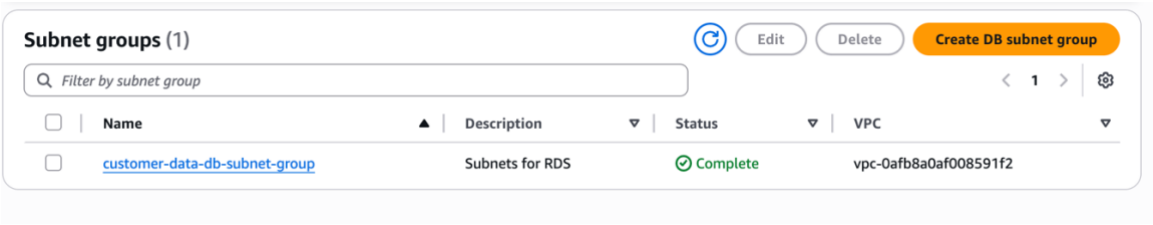


Figure 11: DB subnet group

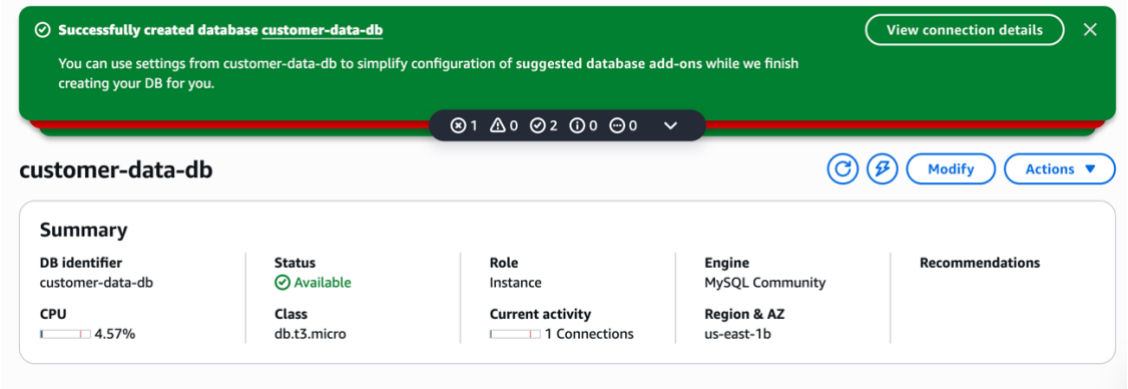


Figure 12: RDS Instance

Implementation Steps

The implementation started by creating the networking environment, including the VPC, subnets, Internet Gateway, and route table. The public subnet was configured to allow internet access through the Internet Gateway.

Next, security groups were created to control traffic between components. The EC2 security group allows SSH access, while the RDS security group allows database access only from the EC2 security group.

An EC2 instance was launched in the public subnet to act as the processing layer. A DB subnet group was created using the private subnets, and an RDS instance was deployed within those subnets to ensure it remains private.

S3 is used as an object storage service for storing raw customer data files and potential exports. In a production environment, the EC2 instance would access S3 through an IAM role. In the AWS

Learner Lab environment, S3 access was validated through the lab terminal because of permission limitations.

A sample customer data file was successfully uploaded to and retrieved from the S3 bucket, confirming that object storage is functioning correctly. In a production environment, access to S3 would be managed through IAM roles attached to EC2 instances instead of relying on preconfigured credentials.

Problems and Solutions

During the implementation, an issue was encountered when attempting to install the MySQL client on the EC2 instance using the yum package manager. The installation failed because the instance was running Amazon Linux 2023, which does not include the mysql package in the default repositories.

This issue was diagnosed based on the error message returned by the package manager. The problem was resolved by installing a MariaDB compatible client instead, which provides the required MySQL functionality.

This experience highlighted the importance of understanding differences between Linux distributions and adapting commands accordingly.

Validation

The system was validated through SSH access, S3 interaction, and database connectivity testing.

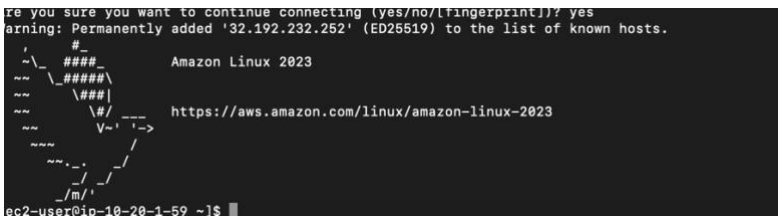


Figure 13: SSH connection to EC2

Next, the EC2 instance was used to connect to the RDS database using the MySQL client. A test database and a customers table were created. Sample customer records were inserted into the table, and a query was executed to retrieve the data successfully.

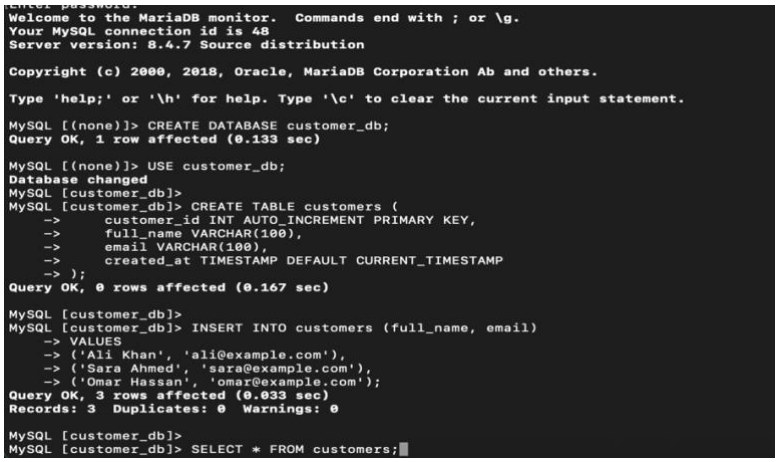


Figure 14: Database connection

Figure 15: Query execution

These tests confirmed that the EC2 instance can communicate with both the S3 bucket and the RDS database, and that the architecture functions as intended.

Phase 2 Implementation and Validation

The goal of Phase 2 was to improve the operational visibility, governance, monitoring, and maintainability of the existing customer data processing environment.

The Phase 1 architecture was first audited and partially restored due to the temporary nature of the AWS Academy Learner Lab environment, where some compute resources did not persist between sessions. Existing networking components, security groups, and storage resources remained available and were reused during the restoration process.

Governance and Resource Tagging

Resource tagging was implemented across major AWS resources to improve governance, organization, filtering, and cost visibility. Consistent tags were added to EC2, RDS, S3, and VPC resources.

The following tags were applied:

- Project = CustomerDataProcessing
- Environment = Development
- Owner = Maab
- Course = CloudOperations

Tagging improves operational management and reflects common enterprise governance practices.

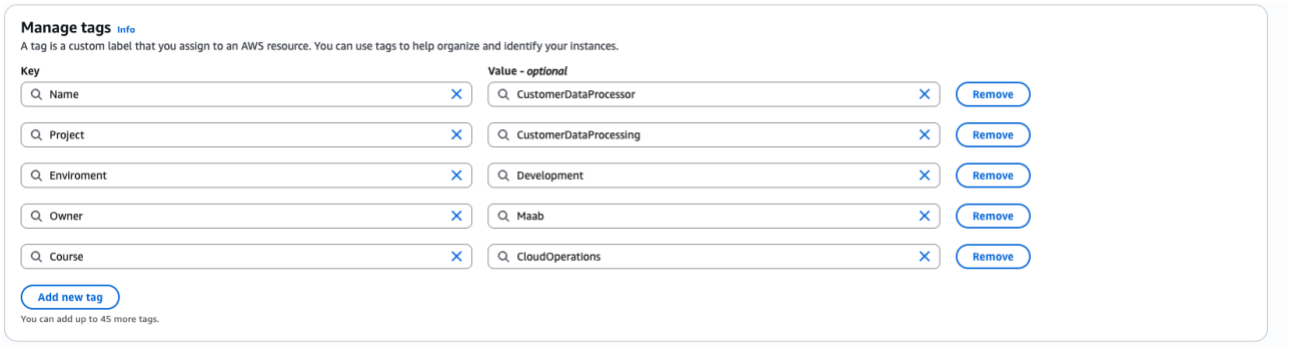


Figure 16: EC2 Resource Tags

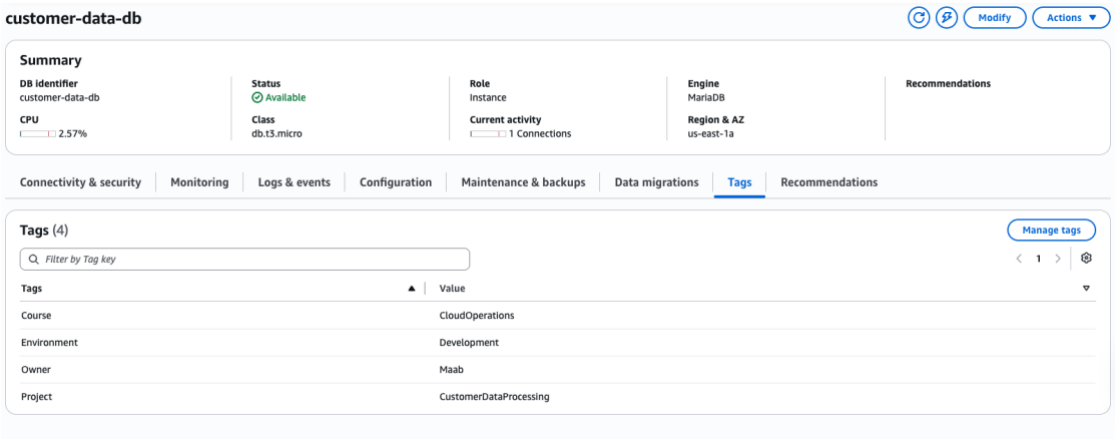


Figure 17: RDS Resource Tags

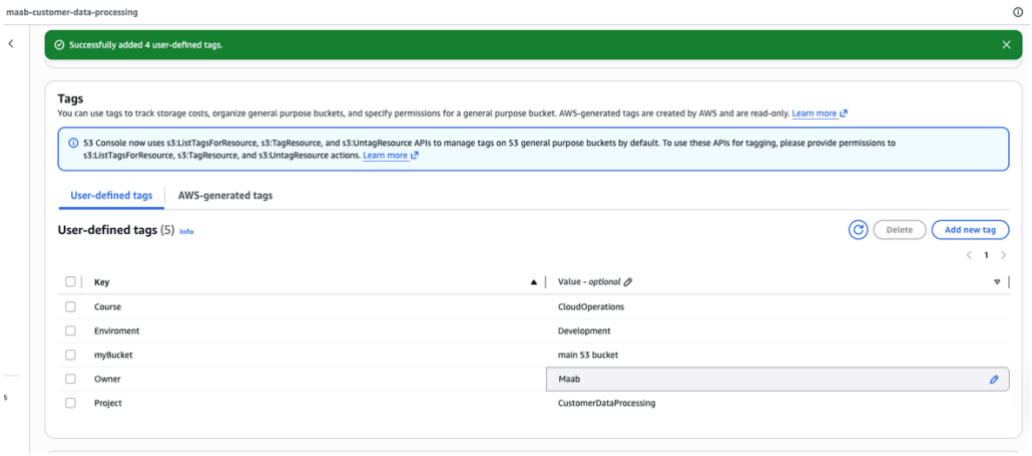


Figure 18: S3 Resource Tags

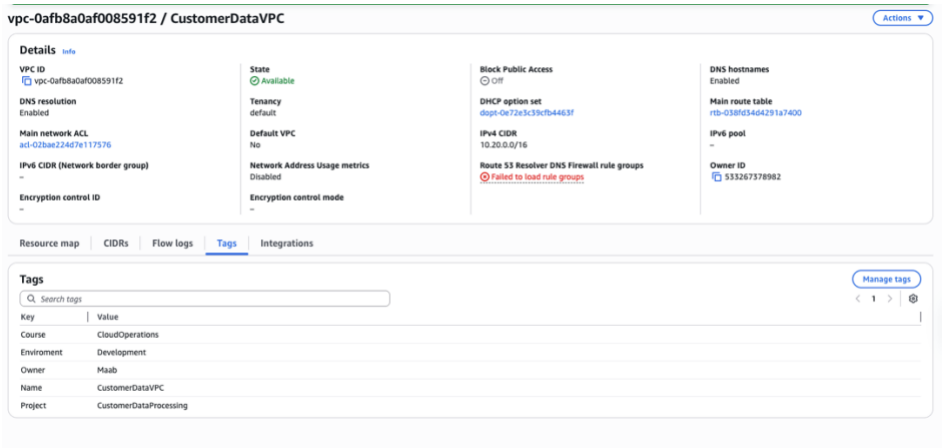


Figure 19: VPC Resource Tags

CloudWatch Monitoring and SNS Alerting

Amazon CloudWatch was implemented to monitor infrastructure metrics and improve operational visibility. A CloudWatch alarm was configured for the EC2 instance to monitor CPU utilization levels.

An Amazon SNS topic was created to send email notifications whenever the alarm threshold was exceeded. The alarm was configured to trigger when EC2 CPU utilization exceeded 70%.

To validate the monitoring configuration, artificial CPU load was generated on the EC2 instance using Linux stress commands. The CloudWatch alarm successfully transitioned into the ALARM state, and SNS email notifications were received successfully.

This implementation demonstrates proactive monitoring and alerting practices used in production cloud environments.

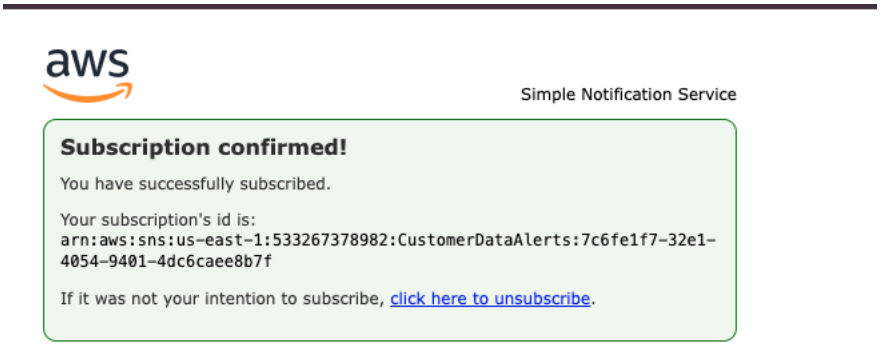


Figure 20: SNS Topic Subscription

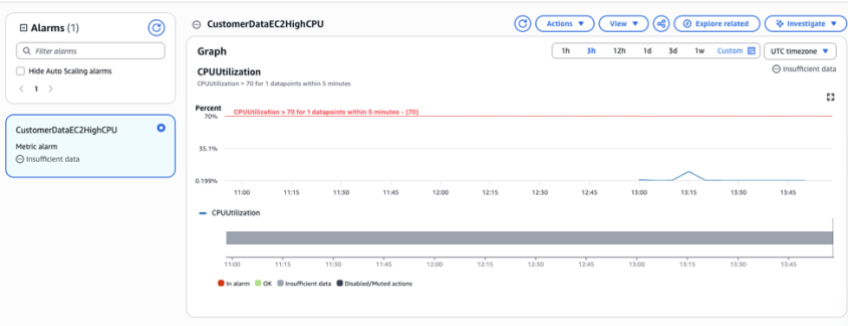


Figure 21: CloudWatch CPU Alarm

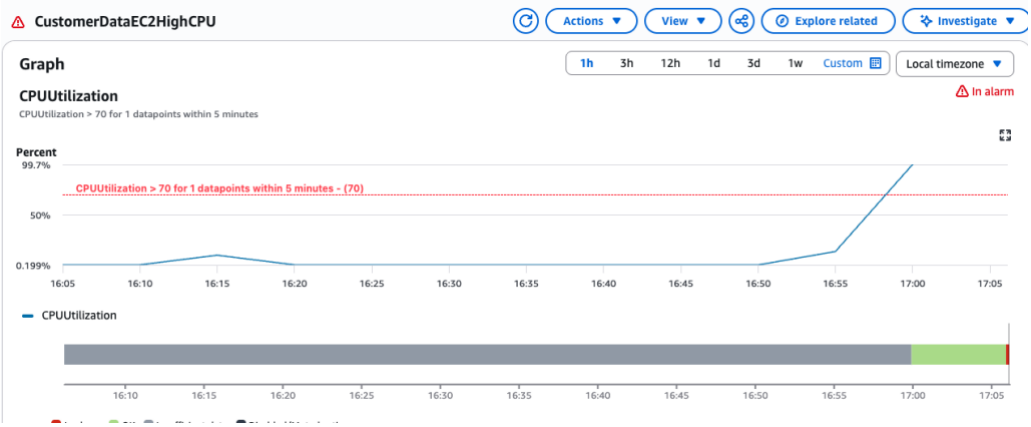
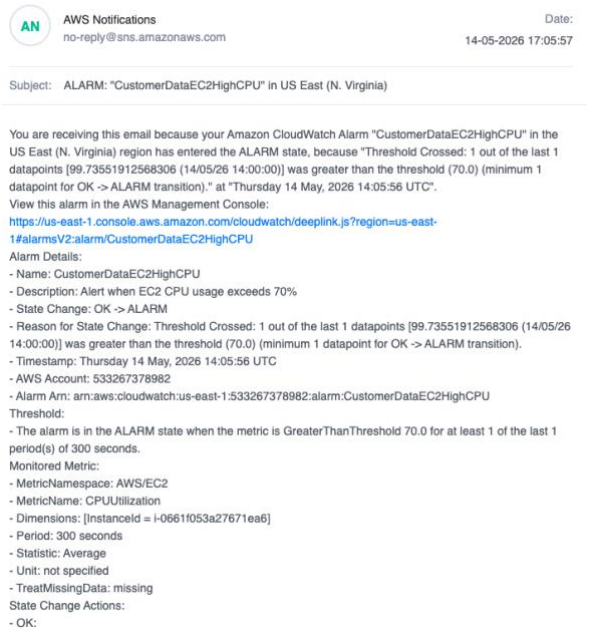


Figure 22: CloudWatch Alarm in ALARM State



AN

AWS Notifications
no-reply@sns.amazonaws.com

Date:
14-05-2026 17:05:57

Subject: ALARM: "CustomerDataEC2HighCPU" in US East (N. Virginia)

You are receiving this email because your Amazon CloudWatch Alarm "CustomerDataEC2HighCPU" in the US East (N. Virginia) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [99.73551912568306 (14/05/26 14:00:00)] was greater than the threshold (70.0) (minimum 1 datapoint for OK -> ALARM transition)." at "Thursday 14 May, 2026 14:05:56 UTC".
View this alarm in the AWS Management Console:
https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/CustomerDataEC2HighCPU
Alarm Details:
- Name: CustomerDataEC2HighCPU
- Description: Alert when EC2 CPU usage exceeds 70%
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [99.73551912568306 (14/05/26 14:00:00)] was greater than the threshold (70.0) (minimum 1 datapoint for OK -> ALARM transition).
- Timestamp: Thursday 14 May, 2026 14:05:56 UTC
- AWS Account: 533267378982
- Alarm Arn: arn:aws:cloudwatch:us-east-1:533267378982:alarm:CustomerDataEC2HighCPU
Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanThreshold 70.0 for at least 1 of the last 1 period(s) of 300 seconds.
Monitored Metric:
- MetricNamespace: AWS/EC2
- MetricName: CPUUtilization
- Dimensions: [InstanceId = i-0661f053a27671ea6]
- Period: 300 seconds
- Statistic: Average
- Unit: not specified
- TreatMissingData: missing
State Change Actions:
- OK:

Figure 23: SNS Email Notification

CloudWatch Dashboard

A centralized CloudWatch dashboard was created to visualize operational metrics for the deployed environment. The dashboard includes:

- EC2 CPU utilization
- EC2 network traffic
- RDS CPU utilization

The dashboard provides a single operational view of the environment and improves monitoring efficiency.

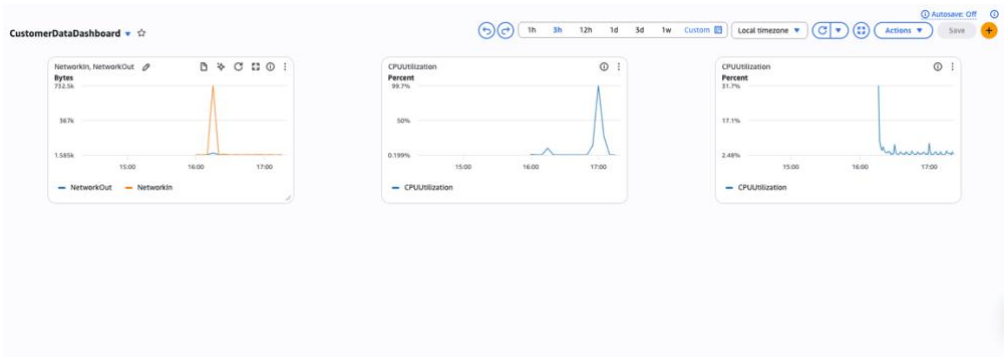


Figure 24: CloudWatch Dashboard

CloudTrail Logging

AWS CloudTrail was enabled to improve governance, auditing, and traceability of API activity within the AWS environment.

CloudTrail records management events across AWS services, including:

- EC2 operations
- RDS modifications
- VPC changes
- Security group changes
- S3 actions

Validation was performed by generating AWS actions and verifying that the events appeared in CloudTrail Event History.

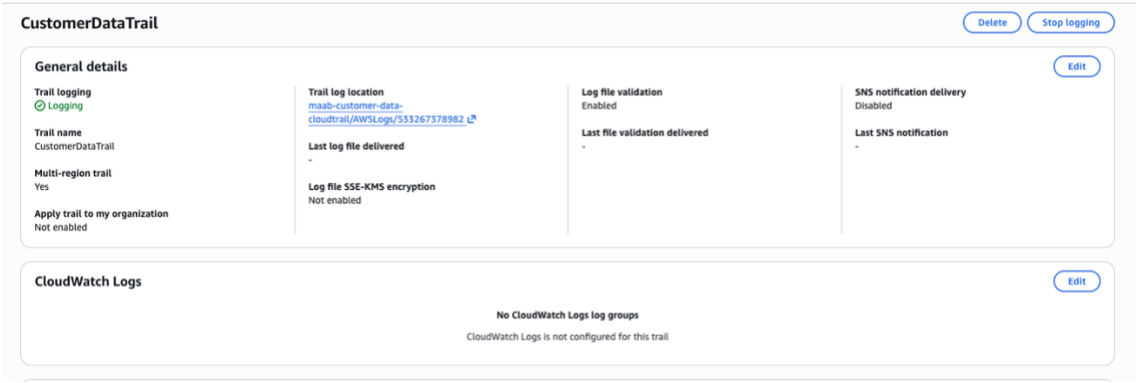


Figure 25: CloudTrail Configuration

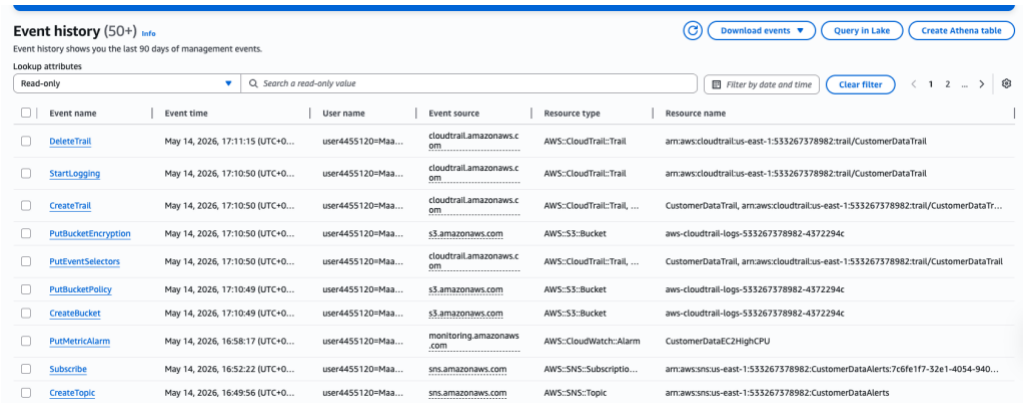


Figure 26: CloudTrail Event History

S3 Lifecycle Policy

An S3 lifecycle policy was implemented to improve storage governance and cost optimization. The lifecycle configuration automatically transitions older objects to Glacier Flexible Retrieval storage after 30 days and permanently deletes objects after 120 days.

This implementation reflects real-world cloud storage management practices by reducing long-term storage costs and automating data retention policies.

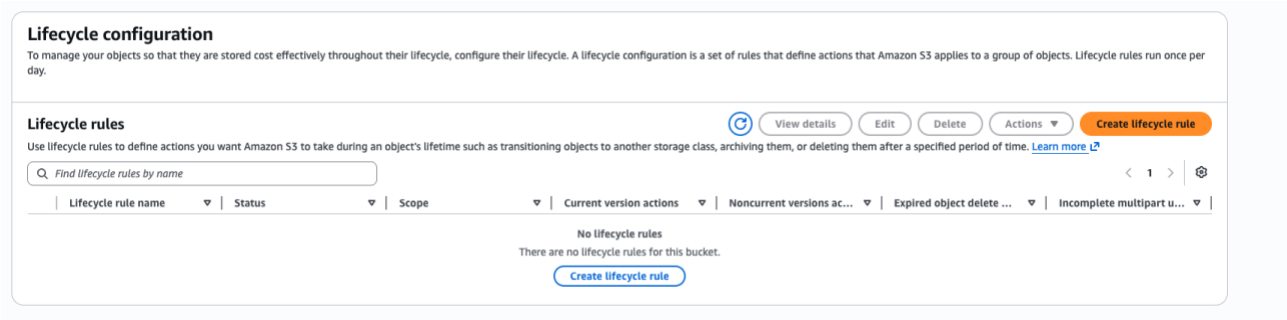


Figure 27: S3 Lifecycle Policy Configuration

Problems and Solutions

During Phase 2 restoration, the database client setup issue from Phase 1 was encountered again because the EC2 instance had been recreated. The database rejected insecure transport connections because secure transport enforcement was enabled by default.

The issue was diagnosed by analyzing the MariaDB client error message. The problem was resolved by establishing the database connection using SSL-enabled client parameters.

Another issue occurred when attempting to install the MySQL client on Amazon Linux 2023. The expected MySQL package was unavailable in the default repositories. This issue was resolved by installing a MariaDB-compatible client package instead.

These troubleshooting scenarios improved understanding of cloud environment compatibility issues, Linux package management differences, and secure database connectivity requirements.

Validation

Phase 2 functionality was validated through multiple tests:

- Resource tags were successfully applied and visible across services
- CloudWatch alarms successfully detected high CPU utilization
- SNS email notifications were delivered successfully
- CloudWatch dashboards displayed operational metrics correctly
- CloudTrail successfully recorded AWS API activity
- S3 lifecycle policy configuration was successfully applied

The validation confirmed that the environment now includes governance, monitoring, logging, and operational visibility features on top of the original architecture.

Enhancements

Several architectural and operational improvements could further enhance the environment.

From an architectural perspective, the solution could be improved by introducing an Application Load Balancer and Auto Scaling Group to improve availability and scalability. Infrastructure as Code solutions such as Terraform or OpenTofu could also be introduced to automate environment deployment and improve reproducibility.

IAM roles were explored as part of the governance improvements, but implementation was limited by AWS Academy Learner Lab permission restrictions. In a production environment, IAM roles with least-privilege access policies would be fully implemented to securely manage EC2 access to AWS services such as S3. Additional CloudWatch alarms and CloudWatch Logs integration could provide deeper operational visibility.

Security could also be strengthened by implementing:

- IAM least privilege policies
- encrypted S3 buckets
- automated backup strategies
- VPC flow logs

- stricter inbound security group rules
- private EC2 deployment through bastion architecture

Cost optimization improvements could include:

- automated shutdown schedules
- storage lifecycle policies
- rightsizing analysis
- reserved instance planning

These improvements would make the environment more scalable, secure, and production-ready.

Conclusions

This project successfully demonstrated the design, deployment, governance, and operational monitoring of a customer data processing system on AWS.

Phase 1 focused on building the core cloud infrastructure using EC2, RDS, S3, and VPC networking components. The environment was validated through successful SSH access, database connectivity, S3 interaction, and SQL query execution.

Phase 2 extended the architecture with cloud governance and operational management capabilities. Monitoring, alerting, logging, and tagging were implemented using CloudWatch, SNS, and CloudTrail. These additions improved operational visibility, governance, and auditing capabilities while reflecting real-world cloud operations practices.

One of the most valuable aspects of the project was troubleshooting and adapting to real operational constraints within the AWS Academy Learner Lab environment. Resource persistence limitations, package compatibility issues, and secure database connection requirements provided practical cloud engineering experience beyond simple deployment

tasks.

If the project were continued further, Infrastructure as Code, scaling solutions, and stronger security automation would be implemented to move the environment closer to a production-grade architecture.

The project was also designed with cost-awareness in mind due to the AWS Academy Learner Lab budget limitations and temporary resource constraints.

Overall, the project provided valuable hands-on experience in AWS infrastructure deployment, troubleshooting, governance, and cloud operations practices.

Appendix

Appendix A: CLI Commands Used

The following AWS CLI and system commands were used during the implementation:

- SSH connection to EC2:

ssh -i labsuser.pem ec2-user@<public-ip>

- Connect to RDS from EC2:

mysql -h <rds-endpoint> -u admin --ssl -p

- Create database and table:

```
CREATE DATABASE customer_data;
USE customer_data;
CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```

- Insert sample data:

```
INSERT INTO customers (name, email)
VALUES ('Alice', 'alice@email.com'),
            ('Bob', 'bob@email.com');
            ('Sara', 'sara@email.com');
```

- Query data:

```
SELECT * FROM customers;
```

- Upload file to S3:

aws s3 cp customer.csv s3://<bucket-name>/

- List files in S3:

aws s3 ls s3://<bucket-name>/

These commands demonstrate the interaction between EC2, RDS, and S3 services.

Appendix B: GitHub Repository

The full implementation, including CLI commands and documentation, is available in the GitHub repository: https://github.com/maab-osman/customer-data-processing-system-aws

Appendix C: Architecture Diagram

The architecture diagram was created using draw.io and represents the system components and their relationships within the AWS environment.

Appendix D: Screenshots

Screenshots included in the report provide evidence of:

- VPC and subnet configuration
- Internet Gateway and route table setup
- Security group rules
- EC2 and RDS deployment
- S3 storage and file upload
- Successful SSH and database connection
- Query execution results
- Resource tagging
- CloudWatch alarm configuration
- SNS subscription and email notification
- CloudWatch dashboard
- CloudTrail configuration and event history

These screenshots validate that the system was successfully implemented and tested.

References

Amazon Web Services Documentation. (2026). Amazon VPC User Guide.

https://docs.aws.amazon.com/vpc/

Amazon Web Services Documentation. (2026). Amazon EC2 User Guide.

https://docs.aws.amazon.com/ec2/

Amazon Web Services Documentation. (2026). Amazon RDS User Guide.

https://docs.aws.amazon.com/rds/

Amazon Web Services Documentation. (2026). Amazon S3 User Guide.

https://docs.aws.amazon.com/s3/

Amazon Web Services Documentation. (2026). AWS CLI Command Reference.

https://docs.aws.amazon.com/cli/

AWS Academy Learner Lab Instructions. (2026). Course materials and lab guidelines.

Amazon Web Services Documentation. (2026). Amazon CloudWatch User Guide.

https://docs.aws.amazon.com/cloudwatch/

Amazon Web Services Documentation. (2026). Amazon SNS Developer Guide.

https://docs.aws.amazon.com/sns/

Amazon Web Services Documentation. (2026). AWS CloudTrail User Guide.

https://docs.aws.amazon.com/cloudtrail/